Listing 11.7 Rendering SVG and canvas simultaneously

```
function createMap(countries) {
    var projection = d3.geo.mercator().scale(100).translate([250,250]);
    var svgPath = d3.geo.path().projection(projection);
    var canvasPath = d3.geo.path().projection(projection);

    var mapZoom = d3.behavior.zoom()
      .translate(projection.translate())
      .scale(projection.scale())
      .on("zoom", zoomed);

    d3.select("svg").call(mapZoom);

    var g = d3.select("svg");

    g.selectAll("path.sample")
      .data(sampleData)
      .enter()
      .append("path")
      .attr("class", "sample")
      .on("mouseover", function() {d3.select(this).style("fill", "pink")});

    zoomed();

    function zoomed() {
      projection.translate(mapZoom.translate()).scale(mapZoom.scale());

      var context = d3.select("canvas").node().getContext("2d");
      context.clearRect(0,0,500,500);
      canvasPath.context(context);

      context.strokeStyle = "black";
      context.fillStyle = "gray";
      context.lineWidth = "1px";
      for (var x in countries.features) {
        context.beginPath();
        canvasPath(countries.features[x]);
        context.stroke();
        context.fill();
      }

    d3.selectAll("path.sample").attr("d", svgPath);
  };
};
```

We need to instantiate a different d3.geo.path for canvas and for SVG.

Updates the map when it's first created

Draws canvas features with canvasPath

Draws SVG features with svgPath

This allows us to maintain interactivity, such as the mouseover function on our triangles to change any triangle's color to pink when moused over. This approach maximizes performance by rendering any graphics that have no interactivity using HTML5 canvas instead of SVG. As shown in figure 11.5, the appearance produced using this method is virtually identical to that using canvas only or SVG only.

But what if you have massive numbers of elements and you really do want interactivity on all them, but you also want to give the user the ability to pan and drag? In that case, you have to embrace an extension of this mixed-mode rendering. You render in canvas whenever users are interacting in such a way that they can't interact with other elements. In other words, we need to render the triangles in canvas when the map is